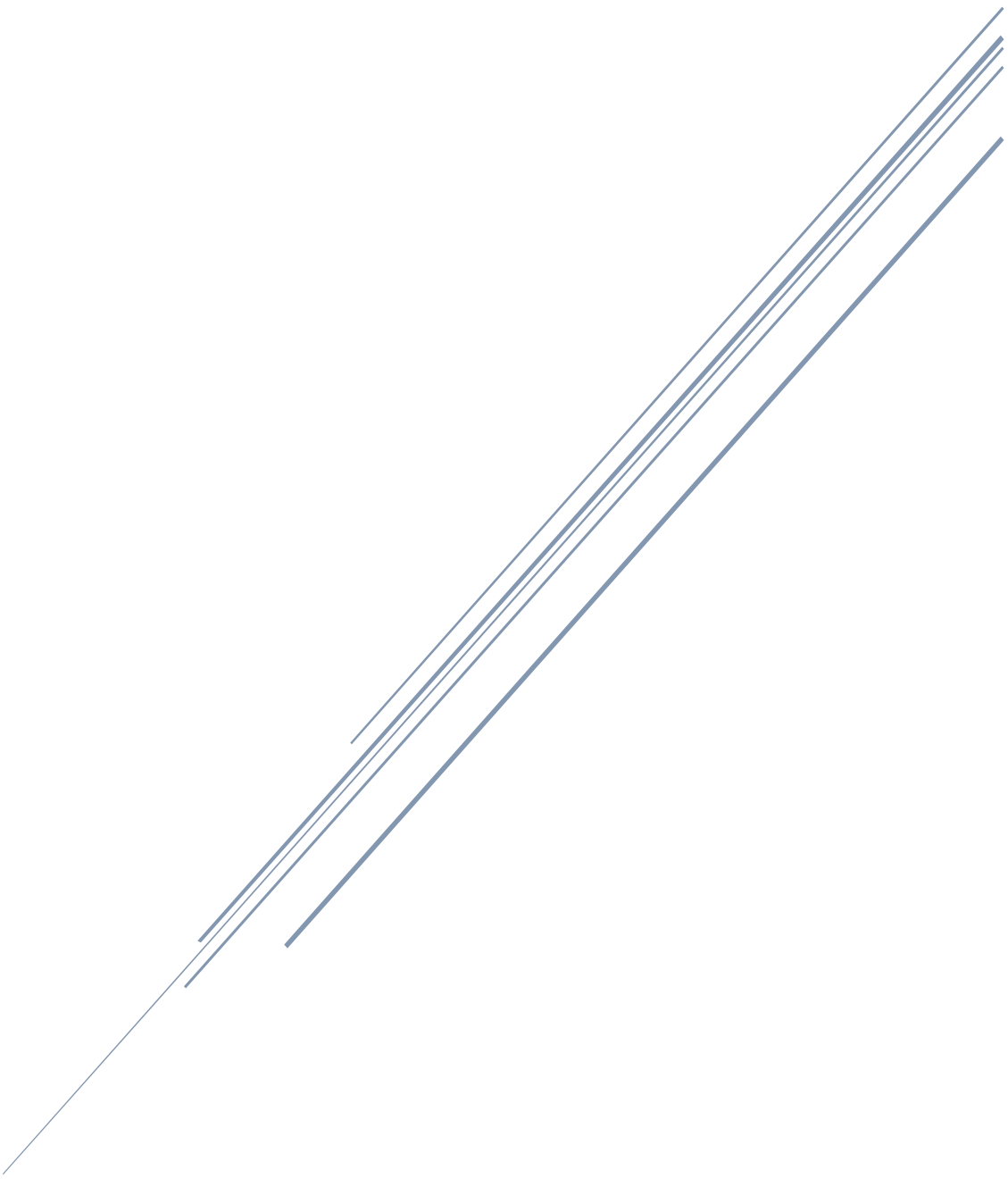
Patient Readmission Prediction Analysis

Leveraging Data Intelligence for Proactive Patient Care



Prepared By: Kapil Kumar Mandawaria

1. Overview: ReadmitGuard

ReadmitGuard is a premier analytics firm dedicated to transforming the healthcare industry through data intelligence. We specialize in predictive modeling and clinical data analysis, providing healthcare facilities with the tools necessary to forecast patient behavior. By identifying trends within complex datasets, ReadmitGuard assists hospitals in moving from reactive treatments to proactive patient management.

1.2 The Clinical Challenge: Patient Readmission

Patient readmission—defined as a patient being admitted to a hospital again within 30 days of discharge—is a critical metric for healthcare quality. High readmission rates often signal gaps in the discharge process or inadequate post-discharge care. For healthcare providers, this results in:

- **Increased Costs:** Penalties from insurance providers and higher operational overhead.
- **Poor Patient Outcomes:** Lower recovery rates and increased risk of hospital-acquired infections.
- **Resource Strain:** Unnecessary occupancy of beds needed for new acute cases.

1.3 Mission Statement

Our mission is to provide healthcare facilities with accurate, reliable, and actionable insights to identify high-risk patients. By leveraging historical medical data, we aim to support clinicians in delivering personalized care plans that ensure patients remain healthy after their initial release, ultimately enhancing the overall quality of care.

1.4 Problem Statement

The objective of this project is to analyze a comprehensive dataset containing patient demographics, medical history, and encounter details to determine the likelihood of readmission. The challenge lies in identifying which variables—ranging from the number of medications to the severity of the diagnosis—serve as the strongest predictors of a patient returning to the facility within the 30-day window.

1.5 Scope of the Report

This report details a full-cycle data analysis project involving:

- **Data Cleaning and Transformation:** Resolving data quality issues, such as converting categorical age buckets into numerical values.
- **Exploratory Data Analysis (EDA):** Using Python and Excel to uncover correlations between inpatient stays, emergency visits, and readmission status.
- **Visual Intelligence:** Developing a Tableau-based executive dashboard to visualize patient risk factors through bubble charts and stacked bar graphs.
- **Actionable Recommendations:** Providing evidence-based strategies to reduce readmission rates based on the observed data patterns.

1.6 Target Stakeholders

- **Hospital Administrators:** Seeking to reduce operational costs and penalties.
- **Clinical Staff:** Looking for data-driven flags to identify high-risk patients during the discharge process.
- **Quality Assurance Teams:** Monitoring the facility's overall performance against healthcare industry benchmarks.

2. Excel Tasks:

Data Exploration:

a) Make a statistical summary of pertinent features like the number of lab procedures, procedures, drugs, and furthermore.

num_lab_procedures		num_procedures		num_medications		number_outpatient		number_emergency		number_inpatient		time_in_hospital	
Mean	43.09564098	Mean	1.339730362	Mean	16.02184423	Mean	0.369357153	Mean	0.197836212	Mean	0.635565906	Mean	4.395986872
Standard Error	0.061673602	Standard Error	0.005347226	Standard Error	0.025477638	Standard Error	0.00397252	Standard Error	0.002916769	Standard Error	0.003958722	Standard Error	0.009357475
Median	44	Median	1	Median	15	Median	0	Median	0	Median	0	Median	4
Mode	1	Mode	0	Mode	13	Mode	0	Mode	0	Mode	0	Mode	3
Standard Deviation	19.67436225	Standard Deviation	1.705806979	Standard Deviation	8.127566209	Standard Deviation	1.267265097	Standard Deviation	0.930472268	Standard Deviation	1.26286329	Standard Deviation	2.985107767
Sample Variance	387.0805299	Sample Variance	2.90977745	Sample Variance	66.05733248	Sample Variance	1.605960825	Sample Variance	0.865778642	Sample Variance	1.594823689	Sample Variance	8.910868383
Kurtosis	-0.245073519	Kurtosis	0.857110302	Kurtosis	3.468154915	Kurtosis	147.9077363	Kurtosis	1191.686726	Kurtosis	20.71939695	Kurtosis	0.85025084
Skewness	-0.236543921	Skewness	1.316414763	Skewness	1.326672134	Skewness	8.832958927	Skewness	22.85558215	Skewness	3.614138992	Skewness	1.133998719
Range	131	Range	6	Range	80	Range	42	Range	76	Range	21	Range	13
Minimum	1	Minimum	0	Minimum	1	Minimum	0	Minimum	0	Minimum	0	Minimum	1
Maximum	132	Maximum	6	Maximum	81	Maximum	42	Maximum	76	Maximum	21	Maximum	14
Sum	4385671	Sum	136339	Sum	1630479	Sum	37588	Sum	20133	Sum	64679	Sum	447362
Count	101766	Count	101766	Count	101766	Count	101766	Count	101766	Count	101766	Count	101766

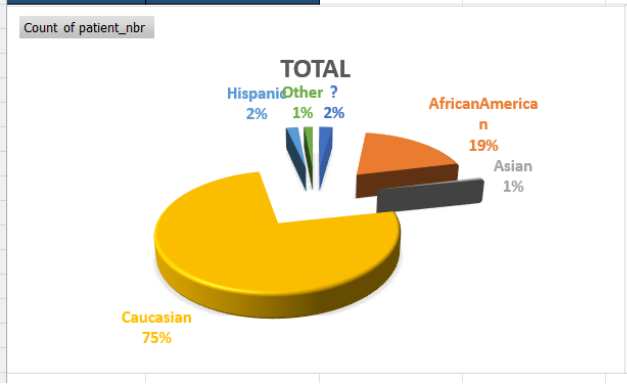
b) Create a report showing the number of readmissions by gender and age group

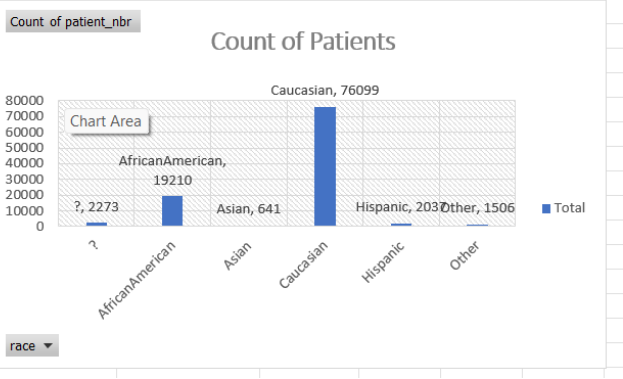
Count of readmitted	Column Labels										
	<30		<30 Total	>30		>30 Total	NO			NO Total	Grand Total
Row Labels	Female	Male		Female	Male		Female	Male	Unknown/Invalid		
[0-10)	1	2	3	13	13	26	69	63		132	161
[10-20)	24	16	40	147	77	224	231	196		427	691
[20-30)	177	59	236	346	164	510	591	320		911	1657
[30-40)	242	182	424	647	540	1187	1273	891		2164	3775
[40-50)	511	516	1027	1685	1593	3278	2615	2765		5380	9685
[50-60)	851	817	1668	3105	2812	5917	4616	5055		9671	17256
[60-70)	1206	1296	2502	4012	3885	7897	5843	6240	1	12084	22483
[70-80)	1635	1434	3069	5108	4367	9475	7242	6280	2	13524	26068
[80-90)	1282	796	2078	3855	2368	6223	5378	3518		8896	17197
[90-100)	223	87	310	600	208	808	1180	495		1675	2793
Grand Total	6152	5205	11357	19518	16027	35545	29038	25823	3	54864	101766

Count of readmitted	Column Labels			
Row Labels	<30	>30	NO	Grand Total
[0-10)	**3**	**26**	**132**	**161**
Female	1	13	69	83
Male	2	13	63	78
[10-20)	**40**	**224**	**427**	**691**
Female	24	147	231	402
Male	16	77	196	289
[20-30)	**236**	**510**	**911**	**1657**
Female	177	346	591	1114
Male	59	164	320	543
[30-40)	**424**	**1187**	**2164**	**3775**
Female	242	647	1273	2162
Male	182	540	891	1613
[40-50)	**1027**	**3278**	**5380**	**9685**
Female	511	1685	2615	4811
Male	516	1593	2765	4874
[50-60)	**1668**	**5917**	**9671**	**17256**
Female	851	3105	4616	8572
Male	817	2812	5055	8684
[60-70)	**2502**	**7897**	**12084**	**22483**
Female	1206	4012	5843	11061
Male	1296	3885	6240	11421

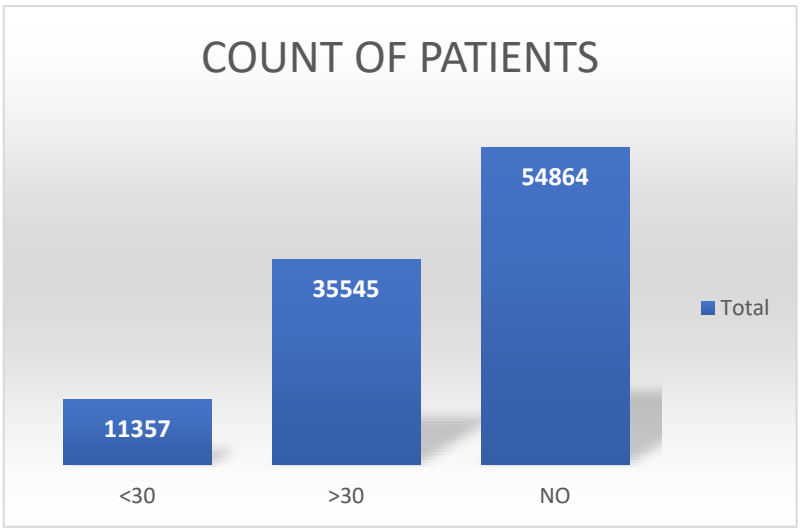
c) Show race distribution in a chart

Row Labels	Count of patient_nbr
?	2273
AfricanAmerican	19210
Asian	641
Caucasian	76099
Hispanic	2037
Other	1506
Grand Total	101766





d) Plot count of patients for each category of readmitted



Row Labels	Count of patient_nbr
<30	11357
>30	35545
NO	54864
Grand Total	101766

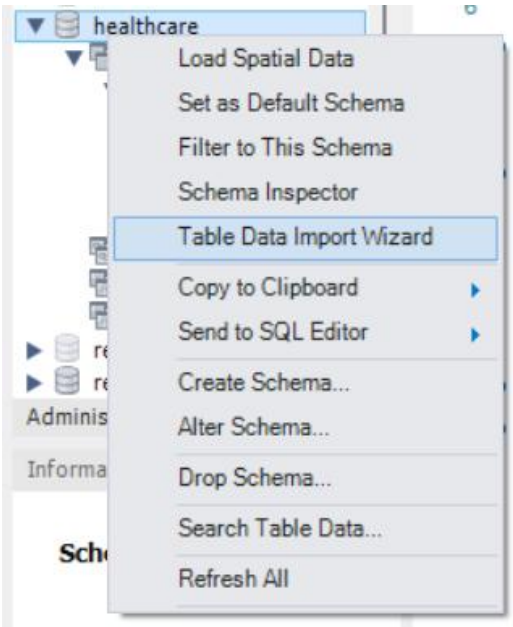
<30	Readmittance Before 30 days
>30	Readmittance After 30 days
No	No Readmittance

3. SQL Tasks:

Data Loading:

- Create a schema named **healthcare**, set it as the default schema, and create tables using **diabetic_data.csv.**
- The dataset was successfully created and imported into MySQL using the **Table Data Import Wizard**. This process was initiated by right-clicking on the newly created schema named **healthcare**, after which the table data was loaded into the database environment for further analysis

```
# Create a schema named healthcare

CREATE SCHEMA healthcare;

# set healthcare as the default schema, and create tables using diabetic_data.csv

USE healthcare;
```




```
SELECT * FROM diabetic_data;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	payer_code	medical_spe
2278392	8222157	Caucasian	Female	[0-10)	?	6	25	1	1	?	Pediatrics-En
149190	55629189	Caucasian	Female	[10-20)	?	1	1	7	3	?	?
64410	86047875	AfricanAmerican	Female	[20-30)	?	1	1	7	2	?	?
500364	82442376	Caucasian	Male	[30-40)	?	1	1	7	2	?	?
16680	42519267	Caucasian	Male	[40-50)	?	1	1	7	1	?	?
35754	82637451	Caucasian	Male	[50-60)	?	2	1	2	3	?	?
55842	84259809	Caucasian	Male	[60-70)	?	3	1	2	4	?	?
63768	114882984	Caucasian	Male	[70-80)	?	1	1	7	5	?	?

Tasks:

a. Calculate the total number of patient encounters in the healthcare dataset

```
# Calculate the total number of patient encounters in the healthcare dataset

SELECT COUNT(encounter_id) AS total_patient_encounters FROM diabetic_data;
SELECT COUNT(patient_nbr) AS total_patient FROM diabetic_data;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_patient_encounters
100100

b. Identify the top 10 most frequent diagnoses in the dataset

```sql
# Identify the top 10 most frequent diagnoses in the dataset

SELECT diagnosis, COUNT(*) AS frequency
FROM (
    SELECT diag_1 AS diagnosis FROM diabetic_data
    UNION ALL
    SELECT diag_2 FROM diabetic_data
    UNION ALL
    SELECT diag_3 FROM diabetic_data
) AS all_diagnoses
WHERE diagnosis IS NOT NULL
  AND diagnosis NOT IN ('?', '')
GROUP BY diagnosis
ORDER BY frequency DESC
LIMIT 10;
```

diagnosis	frequency
428	17962
250	17705
276	13754
414	12848
401	12278
427	11627
599	6742
496	5912
403	5624
486	5430

```sql
SELECT
  icd9_category,
  COUNT(*) AS frequency
FROM (
  SELECT
    CASE
      WHEN diagnosis LIKE 'V%' OR diagnosis LIKE 'E%' THEN 'Supplementary'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 1 AND 139 THEN 'Infectious'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 140 AND 239 THEN 'Neoplasms'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 240 AND 279 THEN 'Endocrine/Metabolic'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 280 AND 289 THEN 'Blood/Immune'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 290 AND 319 THEN 'Mental Disorders'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 320 AND 389 THEN 'Nervous System'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 390 AND 459 THEN 'Circulatory System'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 460 AND 519 THEN 'Respiratory System'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 520 AND 579 THEN 'Digestive System'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 580 AND 629 THEN 'Genitourinary System'
```

```
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 780 AND 799 THEN 'Symptoms'
      WHEN CAST(diagnosis AS DECIMAL(5,2)) BETWEEN 800 AND 999 THEN 'Injury/Poisoning'
      ELSE 'Other'
    END AS icd9_category
  FROM (
    SELECT diag_1 AS diagnosis FROM diabetic_data
    UNION ALL
    SELECT diag_2 FROM diabetic_data
    UNION ALL
    SELECT diag_3 FROM diabetic_data
  ) d
  WHERE diagnosis IS NOT NULL
    AND diagnosis NOT IN ('?', '')
) categorized
GROUP BY icd9_category
ORDER BY frequency DESC  limit 10;
```

Result Grid | Filter Rows:

icd9_category	frequency
Circulatory System	90745
Endocrine/Metabolic	58308
Respiratory System	27264
Genitourinary System	19226
Digestive System	16665
Symptoms	16607
Injury/Poisoning	11194
Skin	8541
Musculoskeletal	8431
Mental Disorders	8034

c. Calculate the average length of hospital stay for each admission type

```
28  # Calculate the average length of hospital stay for each admission type
29
30  select admission_type_id, AVG(time_in_hospital) as average_hospital_stay
31  from diabetic_data
32  group by admission_type_id
33  order by admission_type_id;
34
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

admission_type_id	average_hospital_stay
1	4.3786
2	4.5504
3	4.1197
4	3.2000
5	3.9438
6	4.5012
7	4.8571
8	3.0732

d. Determine the number of readmitted patients and the percentage of total encounters that they represent

```
47    # Determine the number of readmitted patients and the percentage of total encounters that they represent
48
49    SELECT
50        readmitted,
51        COUNT(*) AS encounter_count,
52        ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM diabetic_data), 4) AS percentage_of_total_encounters
53    FROM diabetic_data
54    WHERE readmitted IN ('<30', '>30')
55    GROUP BY readmitted;
56
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

readmitted	encounter_count	percentage_of_total_encounters
>30	34980	34.9451
<30	11086	11.0749

e. Identify the age distribution of patients

```
56
57    # Identify the age distribution of patients
58
59    SELECT age, COUNT(patient_nbr) AS Number_of_Patients
60    FROM diabetic_data
61    GROUP BY age
62    ORDER BY age;
63
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

age	Number_of_Patients
[0-10)	160
[10-20)	690
[20-30)	1650
[30-40)	3753
[40-50)	9588
[50-60)	17018
[60-70)	22083
[70-80)	25528
[80-90)	16872

Result 12

f. Identify the most common procedures performed during patient encounters

```sql
# Identify the most common procedures performed during patient encounters

SELECT
    num_procedures,
    COUNT(*) AS frequency
FROM diabetic_data
GROUP BY num_procedures
ORDER BY frequency DESC
LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

num_procedures	frequency
0	45915
1	20368
2	12371

g. Calculate the average number of medications prescribed for patients in each age group

```sql
# Calculate the average number of medications prescribed for patients in each age group

SELECT age,  AVG(num_medications) AS average_number_medications
FROM diabetic_data
GROUP BY age
order by age;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

age	average_number_medications
[0-10)	6.1125
[10-20)	8.2710
[20-30)	11.9485
[30-40)	14.0818
[40-50)	15.3835
[50-60)	16.5804
[60-70)	17.1794
[70-80)	16.4307
[80-90)	15.3503

h. Identify the distribution of readmission rates across different payer codes

```sql
# Breakdown the readmission rates for '<30' and '>30' as separate percentages across different payer codes

SELECT
    payer_code,
    COUNT(*) AS total_encounters,
    -- Calculate count and % for readmission under 30 days
    SUM(CASE WHEN readmitted = '<30' THEN 1 ELSE 0 END) AS readmitted_under_30_count,
    ROUND(AVG(CASE WHEN readmitted = '<30' THEN 1.0 ELSE 0.0 END) * 100, 2) AS pct_under_30,
    -- Calculate count and % for readmission over 30 days
    SUM(CASE WHEN readmitted = '>30' THEN 1 ELSE 0 END) AS readmitted_over_30_count,
    ROUND(AVG(CASE WHEN readmitted = '>30' THEN 1.0 ELSE 0.0 END) * 100, 2) AS pct_over_30
FROM diabetic_data
GROUP BY payer_code
ORDER BY total_encounters DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payer_code	total_encounters	readmitted_under_30_count	pct_under_30	readmitted_over_30_count	pct_over_30
?	39368	4487	11.40	13516	34.33
MC	31955	3728	11.67	11818	36.98
HM	6207	636	10.25	2301	37.07
SP	4968	505	10.17	1853	37.30
BC	4618	419	9.07	1291	27.96
MD	3509	413	11.77	1252	35.68
CP	2480	200	8.06	760	30.65
UN	2427	225	9.27	689	28.39
CM	1914	196	10.24	657	34.33
OG	1026	132	12.87	333	32.46
PO	580	42	7.24	142	24.48
DM	546	63	11.54	219	40.11
CH	145	13	8.97	33	22.76
WC	131	5	3.82	24	18.32

4. Python Task:

The diabetic_data.csv file loaded into a pandas DataFrame called data. The first 5 rows of the DataFrame, along with its 50 columns, are displayed above. This gives us a quick overview of the data, including patient encounter information, demographics (race, gender, age), admission details, and medication use.

[1]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as sc
```

[3]:
```python
# Load the dataset
# Import diabetic_data.csv into a pandas DataFrame
data = pd.read_csv(r"C:\Users\kalya\OneDrive\Documents\KAPIL_DATAanalysis\Capstone Project\Healthcare_Management_Datasets\diabetic_data.csv")
data
```

[3]:

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	...	citoglipt
0	2278392	8222157	Caucasian	Female	[0-10)	?	6	25	1	1	...	
1	149190	55629189	Caucasian	Female	[10-20)	?	1	1	7	3	...	
2	64410	86047875	AfricanAmerican	Female	[20-30)	?	1	1	7	2	...	
3	500364	82442376	Caucasian	Male	[30-40)	?	1	1	7	2	...	

EDA Task:

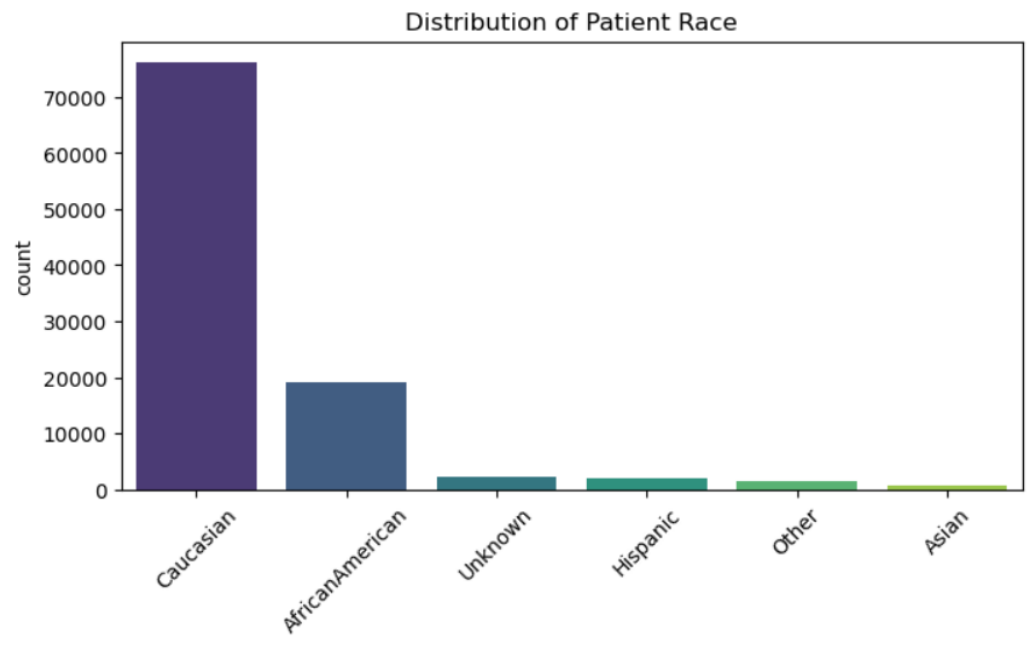
a) Perform descriptive statistical analysis for numerical features

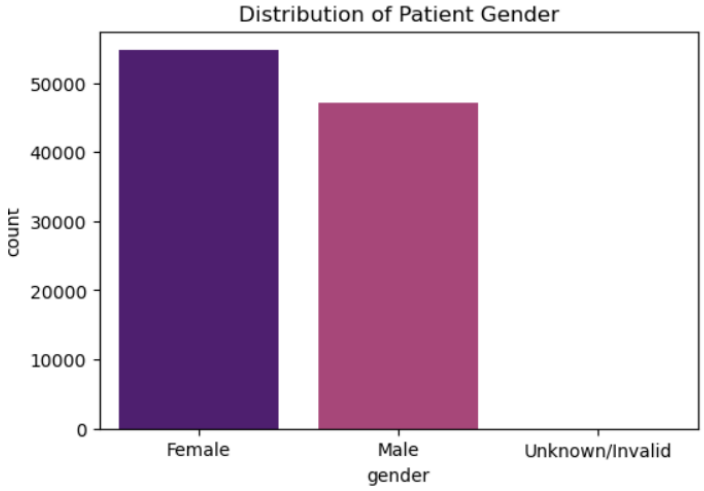
```
# Perform descriptive statistical analysis for numerical features
# 1. Basic Summary Statistics
# This includes count, mean, std, min, 25%, 50% (median), 75%, and max
summary = data.describe()

# 2. Advanced Metrics (Skewness and Kurtosis)
# Essential for fraud detection to identify extreme outliers
skewness = data.skew(numeric_only=True)
kurtosis = data.kurtosis(numeric_only=True)

# 3. Mode Calculation
# (Since .describe() doesn't include mode)
modes = data.mode(numeric_only=True).iloc[0]

# 3. Create a DataFrame for the new metrics
# We convert the Series to DataFrames and transpose so they match the 'summary' structure
extra_metrics = pd.DataFrame({
    'skewness': skewness,
    'kurtosis': kurtosis,
    'mode': modes
}).T

# 4. Use pd.concat instead of .append()
analysis_report = pd.concat([summary, extra_metrics])

print(analysis_report)
```

```
          number_outpatient  number_emergency  number_inpatient  \
count         101766.000000     101766.000000     101766.000000
mean               0.369357          0.197836          0.635566
std                1.267265          0.930472          1.262863
min                0.000000          0.000000          0.000000
25%                0.000000          0.000000          0.000000
50%                0.000000          0.000000          0.000000
75%                0.000000          0.000000          1.000000
max               42.000000         76.000000         21.000000
skewness           8.832959         22.855582          3.614139
kurtosis         147.907736       1191.686726         20.719397
mode               0.000000          0.000000          0.000000

          number_diagnoses
count       101766.000000
mean             7.422607
std              1.933600
min              1.000000
25%              6.000000
50%              8.000000
75%              9.000000
max             16.000000
skewness        -0.876746
kurtosis        -0.079056
mode             9.000000
```

b) Visualize the distribution of categorical features - race and gender

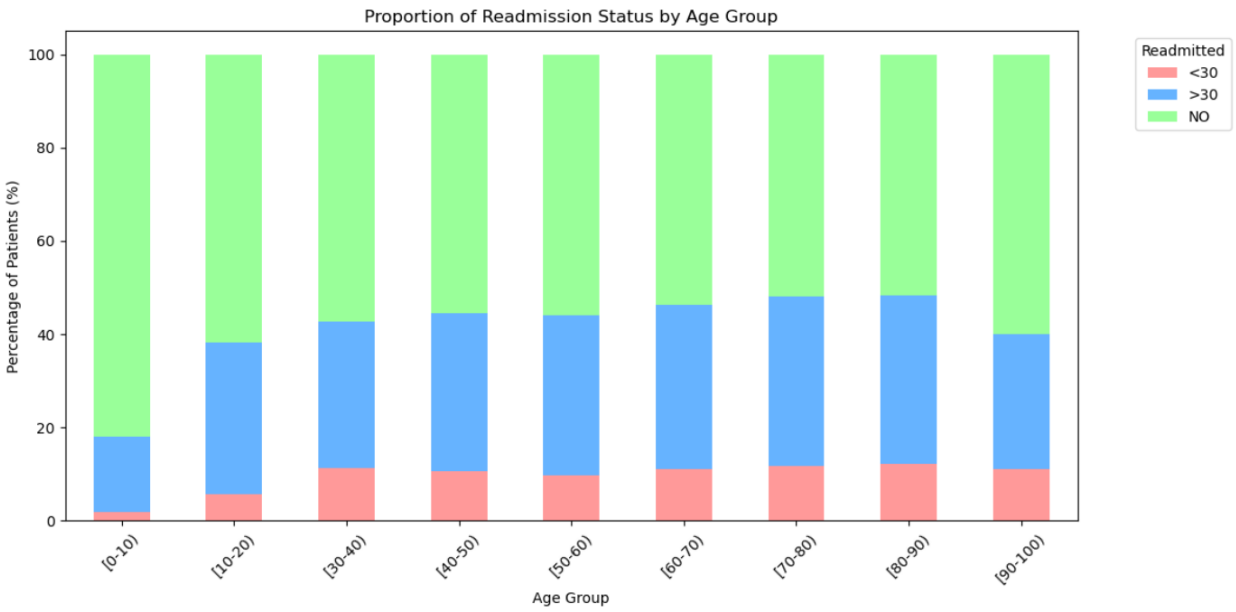
```
# Visualize the distribution of categorical features - race and gender

#  Clean '?' values found in the dataset
# Replacing '?' with 'Unknown' makes the charts more readable.
data['race'] = data['race'].replace('?', 'Unknown')
data['gender'] = data['gender'].replace('?', 'Unknown')

# 4. Visualize Race Distribution
plt.figure(figsize=(8, 4))
# Using order=... ensures bars are sorted by frequency
sns.countplot(data=data, x='race', palette='viridis', order=data['race'].value_counts().index)
plt.title('Distribution of Patient Race')
plt.xticks(rotation=45)
plt.savefig('race_distribution.png')

# 5. Visualize Gender Distribution
plt.figure(figsize=(6, 4))
sns.countplot(data=data, x='gender', palette='magma')
plt.title('Distribution of Patient Gender')
plt.savefig('gender_distribution.png')
```



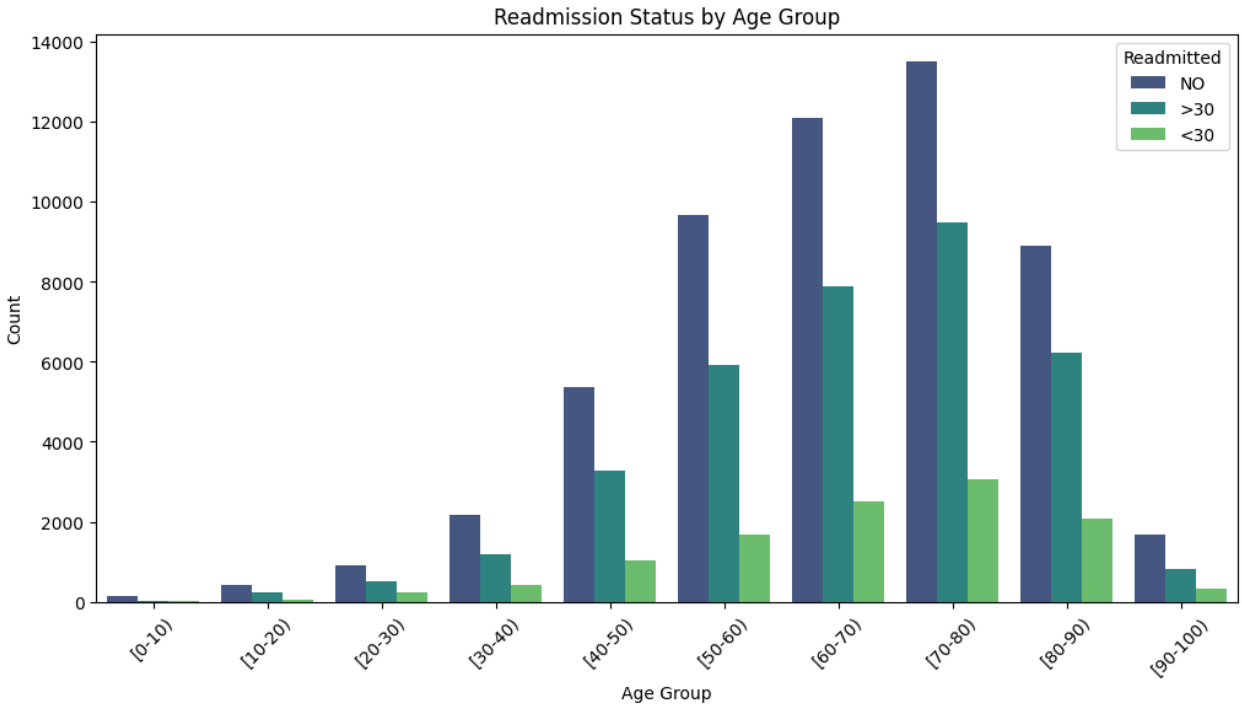


c) Explore the relationship between readmission status and age

```
# task 3: Explore the relationship between readmission status and age
# 3. Create a Crosstab (Table) of Age vs Readmission
age_readmission_table = pd.crosstab(data['age'], data['readmitted'], normalize='index') * 100
# Reorder the index based on age bins
age_readmission_table = age_readmission_table.reindex(age_order)

# 4. Visualize with a Stacked Bar Chart
age_readmission_table.plot(kind='bar', stacked=True, figsize=(12, 6), color=['#ff9999','#66b3ff','#99ff99'])
plt.title('Proportion of Readmission Status by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Percentage of Patients (%)')
plt.legend(title='Readmitted', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('age_vs_readmission.png')

# 5. Statistical Summary Table
print("Readmission Rate (%) by Age Group:")
print(age_readmission_table.round(2))
```




```
# Set the figure size for better readability
plt.figure(figsize=(12, 6))
# Create a count plot for 'age' groups, colored by 'readmitted' status
sns.countplot(data=df, x='age', hue='readmitted', palette='viridis', order=sorted(df['age'].unique()))
# Add a title to the plot
plt.title('Readmission Status by Age Group')
# Add x-axis label
plt.xlabel('Age Group')
# Add y-axis label
plt.ylabel('Count')
# Rotate x-axis labels for better visibility
plt.xticks(rotation=45)
# Add a legend for the 'readmitted' hue
plt.legend(title='Readmitted')
# Display the plot
plt.show()
```



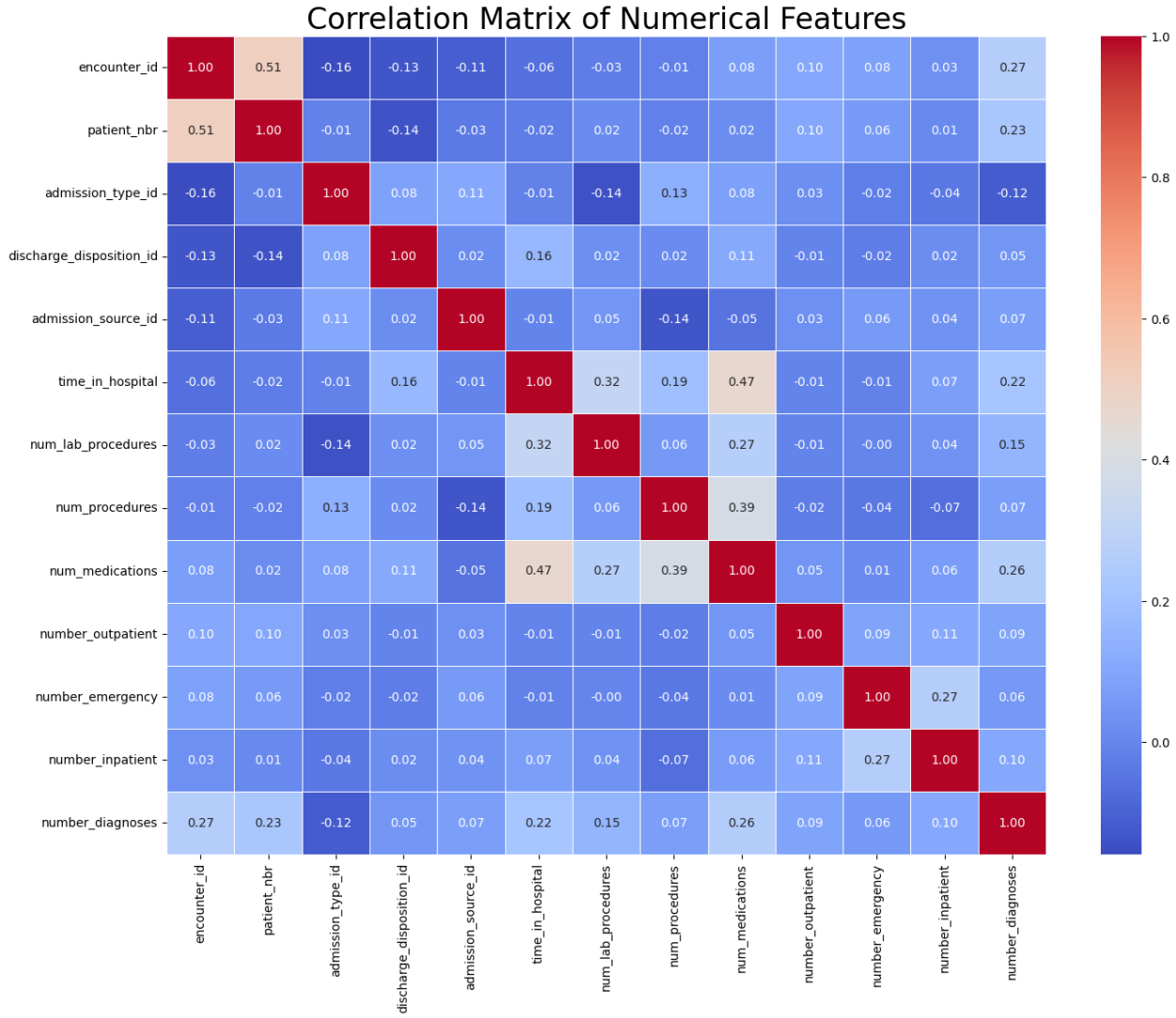
d) Investigate correlations between numerical features

```
[10]: # Task 4: Investigate correlations between numerical features

correlation_matrix = data.select_dtypes([float,int]).corr()
print("Correlation between numerical varables: ")
correlation_matrix
```

```
Correlation between numerical varables:
```

[10]:

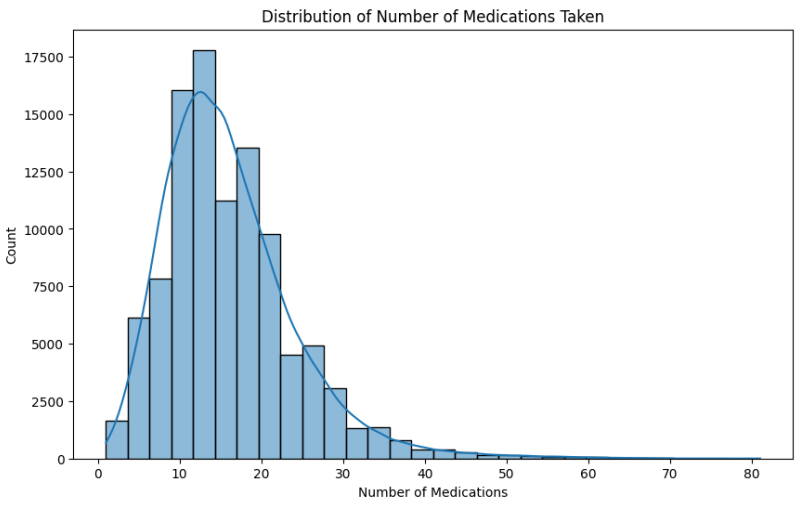
	encounter_id	patient_nbr	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	num_lab_procedures	num_procedu
encounter_id	1.000000	0.512028	-0.158961	-0.132876	-0.112402	-0.062221	-0.026062	-0.0142
patient_nbr	0.512028	1.000000	-0.011128	-0.136814	-0.032568	-0.024092	0.015946	-0.0155
admission_type_id	-0.158961	-0.011128	1.000000	0.083483	0.106654	-0.012500	-0.143713	0.1298
discharge_disposition_id	-0.132876	-0.136814	0.083483	1.000000	0.018193	0.162748	0.023415	0.0159
admission_source_id	-0.112402	-0.032568	0.106654	0.018193	1.000000	-0.006965	0.048885	-0.1354
time_in_hospital	-0.062221	-0.024092	-0.012500	0.162748	-0.006965	1.000000	0.318450	0.1914
num_lab_procedures	-0.026062	0.015946	-0.143713	0.023415	0.048885	0.318450	1.000000	0.0580
num_procedures	-0.014225	-0.015570	0.129888	0.015921	-0.135400	0.191472	0.058066	1.0000
num_medications	0.076113	0.020665	0.079535	0.108753	-0.054533	0.466135	0.268161	0.3857
number_outpatient	0.103756	0.103379	0.026511	-0.008715	0.027244	-0.008916	-0.007602	-0.0248
number_emergency	0.082803	0.062352	-0.019116	-0.024471	0.059892	-0.009681	-0.002279	-0.0381
number_inpatient	0.030962	0.012480	-0.038161	0.020787	0.036314	0.073623	0.039231	-0.0662
number_diagnoses	0.265149	0.226847	-0.117126	0.046891	0.072114	0.220186	0.152773	0.0737

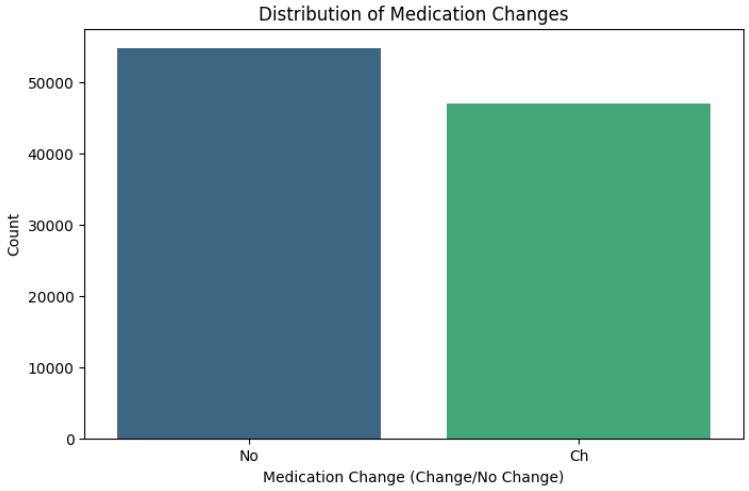


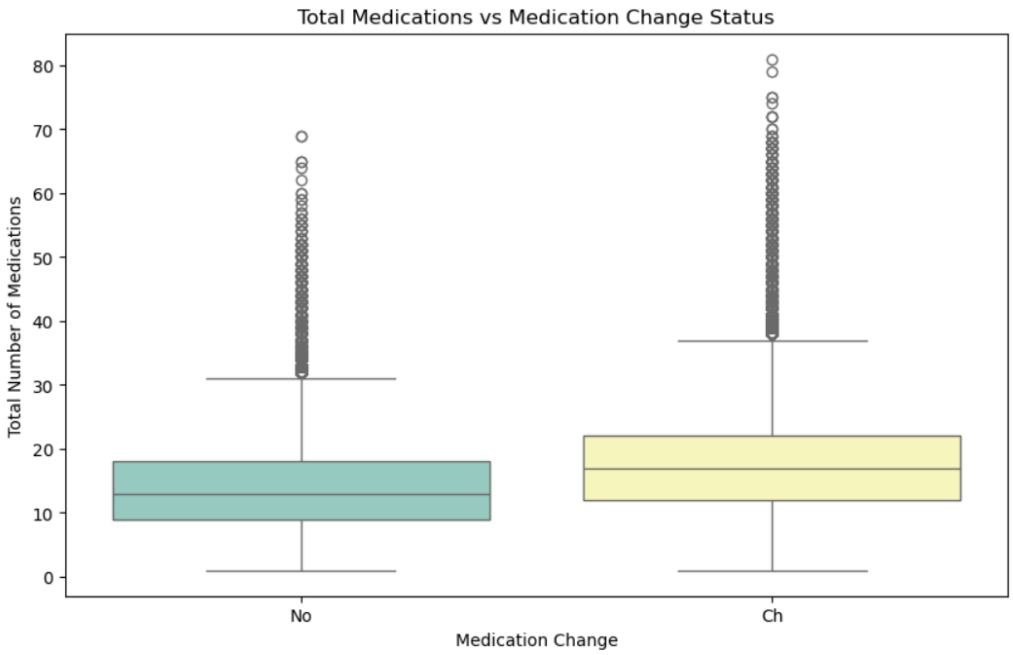
e) Analyze the distribution of medication changes and total medications taken

```
Summary Table: Medications by Change Status
  change        mean  median       std  count
0     Ch   18.187041    17.0  8.631313  47011
1     No   14.162871    13.0  7.164451  54755

Descriptive Statistics for Total Medications:
count    101766.000000
mean         16.021844
std           8.127566
min           1.000000
25%          10.000000
50%          15.000000
75%          20.000000
max          81.000000
Name: num_medications, dtype: float64
```

```python
# Task 5: Analyze the distribution of medication changes and total medications taken
#  Total Medications Taken (num_medications) - Numerical Analysis
print("Descriptive Statistics for Total Medications:")
print(data['num_medications'].describe())

plt.figure(figsize=(10, 6))
sns.histplot(data['num_medications'], kde=True, color='skyblue', bins=30)
plt.title('Distribution of Total Medications Taken')
plt.xlabel('Number of Medications')
plt.ylabel('Frequency')
plt.savefig('num_medications_distribution.png')

# 3. Medication Changes (change) - Categorical Analysis
# 'Ch' indicates a change, 'No' indicates no change
plt.figure(figsize=(8, 6))
sns.countplot(data=data, x='change', palette='pastel')
plt.title('Distribution of Medication Changes (Ch vs No)')
plt.xlabel('Change in Medication')
plt.ylabel('Count')
plt.savefig('medication_change_distribution.png')

# 4. Relationship between Total Medications and Change Status
plt.figure(figsize=(10, 6))
sns.boxplot(data=data, x='change', y='num_medications', palette='Set3')
plt.title('Total Medications vs Medication Change Status')
plt.xlabel('Medication Change')
plt.ylabel('Total Number of Medications')
plt.savefig('med_vs_change_boxplot.png')

# 5. Summary Table
med_summary = data.groupby('change')['num_medications'].agg(['mean', 'median', 'std', 'count']).reset_index()
print("\nSummary Table: Medications by Change Status")
print(med_summary)
```

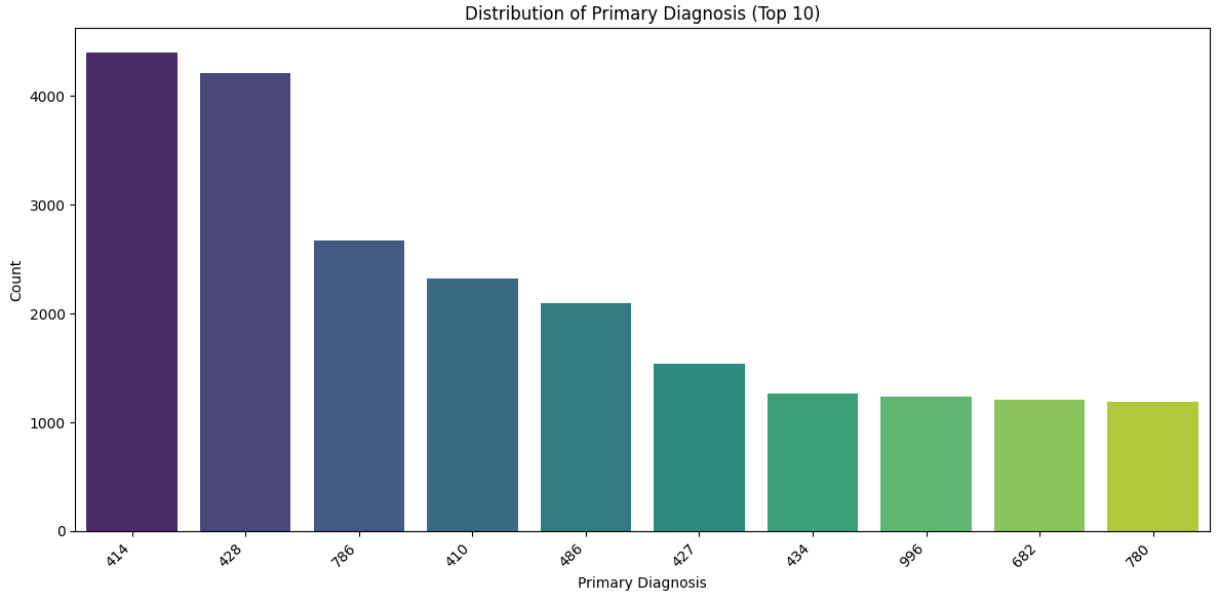


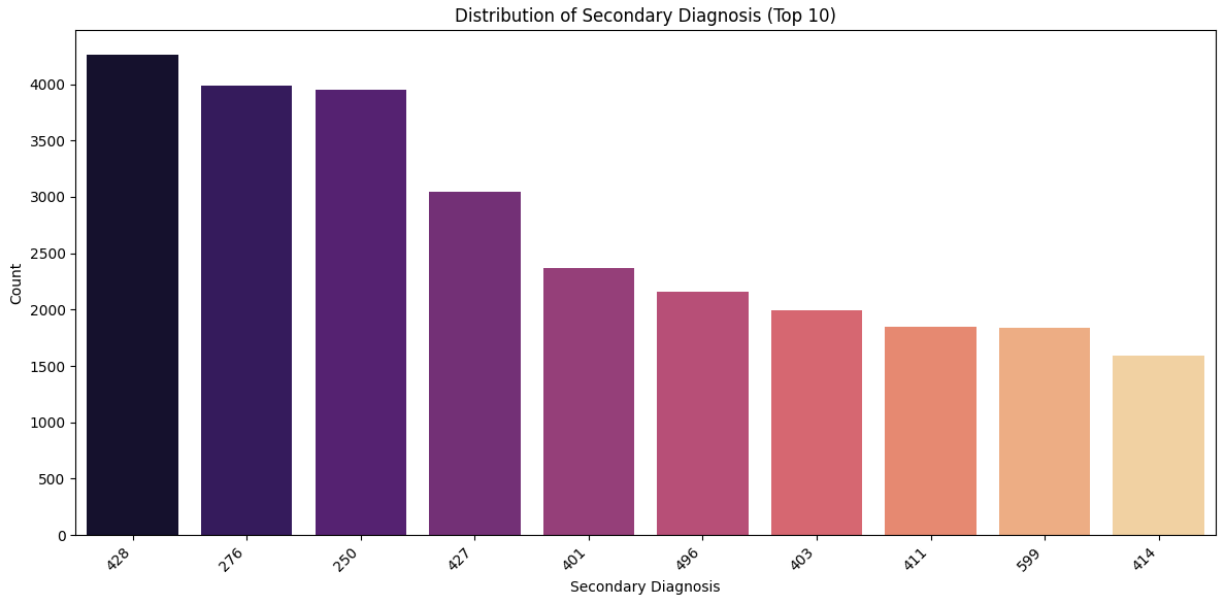


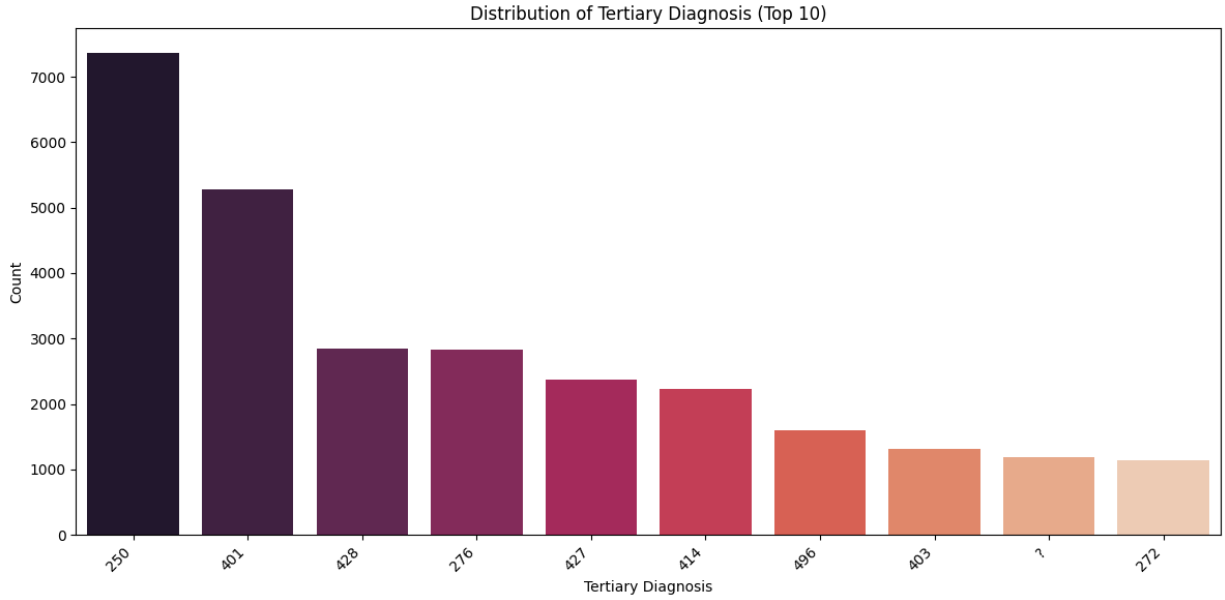


f) Examine the distribution of diagnoses categories

```
# Task 6: Examine the distribution of diagnoses categories
# Create a count plot for the 'diag_1' column
plt.figure(figsize=(12, 6))
sns.countplot(data=data, x='diag_1', palette='viridis', order=data['diag_1'].value_counts().index[:10]) # Display top 10 for readability
plt.title('Distribution of Primary Diagnosis (Top 10)')
plt.xlabel('Primary Diagnosis')
plt.ylabel('Count')
# Rotate x-axis labels for better visibility
plt.xticks(rotation=45, ha='right')
# Display the plot
plt.tight_layout()
plt.show()

# Set count plot for the 'diag_2' column
plt.figure(figsize=(12, 6))
# Create a count plot for the 'diag_2' column
sns.countplot(data=data, x='diag_2', palette='magma', order=data['diag_2'].value_counts().index[:10]) # Display top 10 for readability
plt.title('Distribution of Secondary Diagnosis (Top 10)')
plt.xlabel('Secondary Diagnosis')
plt.ylabel('Count')
# Rotate x-axis labels for better visibility
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Create a count plot for the 'diag_3' column
plt.figure(figsize=(12, 6))
sns.countplot(data=data, x='diag_3', palette='rocket', order=data['diag_3'].value_counts().index[:10]) # Display top 10 for readability
# Add a title to the plot
plt.title('Distribution of Tertiary Diagnosis (Top 10)')
plt.xlabel('Tertiary Diagnosis')
plt.ylabel('Count')
```


Distribution of Primary Diagnosis (Top 10)
Count
0
1000
2000
3000
4000
414
428
786
410
486
427
434
996
682
780
Primary Diagnosis

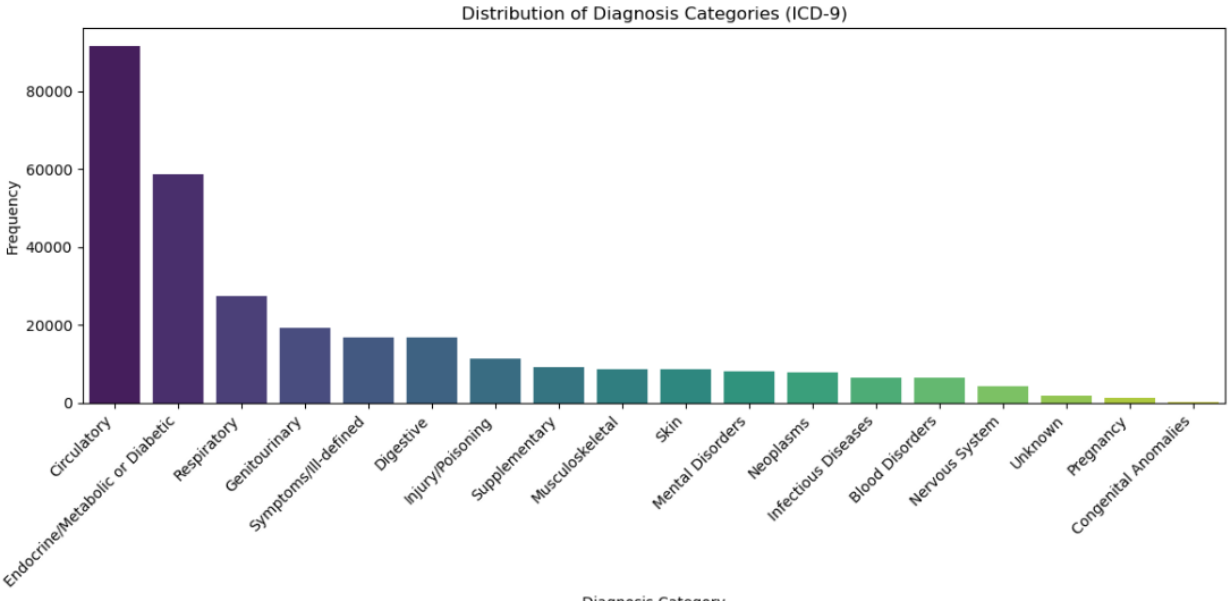

Distribution of Secondary Diagnosis (Top 10)
Count
0
500
1000
1500
2000
2500
3000
3500
4000
428
276
250
427
401
496
403
411
599
414
Secondary Diagnosis


Distribution of Tertiary Diagnosis (Top 10)
Count
0
1000
2000
3000
4000
5000
6000
7000
250
401
428
276
427
414
496
403
?
272
Tertiary Diagnosis

For better clarification we can map the diagnoses number based on the ICD-9 Category Mapping

```
# Function to map ICD-9 codes to categories
def map_icd9_category(code):
    try:
        code = str(code)
        if code.startswith("V") or code.startswith("E"):
            return "Supplementary"
        code = int(float(code))  # convert to integer
    except:
        return "Unknown"

    if 1 <= code <= 139:
        return "Infectious Diseases"
    elif 140 <= code <= 239:
        return "Neoplasms"
    elif 240 <= code <= 279:
        return "Endocrine/Metabolic or Diabetic"
    elif 280 <= code <= 289:
        return "Blood Disorders"
    elif 290 <= code <= 319:
        return "Mental Disorders"
    elif 320 <= code <= 389:
        return "Nervous System"
    elif 390 <= code <= 459:
        return "Circulatory"
    elif 460 <= code <= 519:
        return "Respiratory"
    elif 520 <= code <= 579:
        return "Digestive"
    elif 580 <= code <= 629:
        return "Genitourinary"
    elif 630 <= code <= 679:
        return "Pregnancy"
    elif 680 <= code <= 709:
        return "Skin"
    elif 710 <= code <= 739:
        return "Musculoskeletal"
    elif 740 <= code <= 759:
        return "Congenital Anomalies"
    elif 760 <= code <= 779:
        return "Perinatal Conditions"
    elif 780 <= code <= 799:
        return "Symptoms/Ill-defined"
    elif 800 <= code <= 999:
        return "Injury/Poisoning"
    else:
        return "Unknown"
```

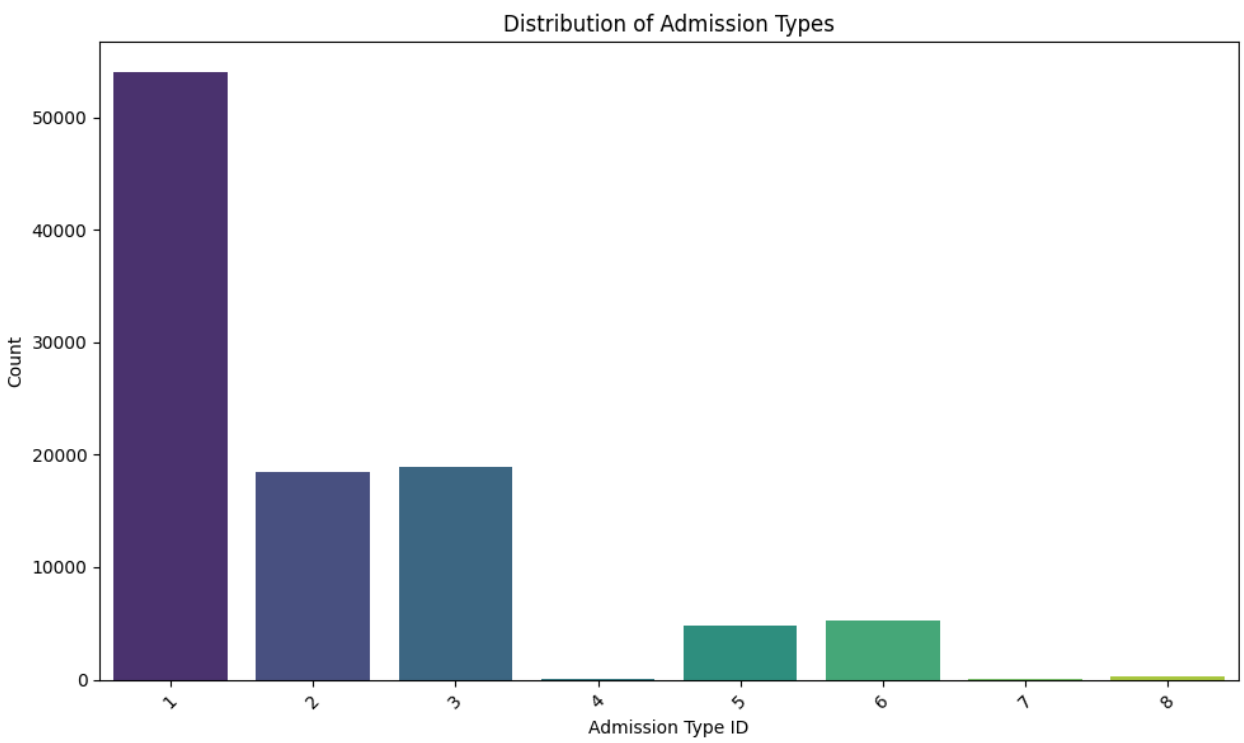
```
# Apply mapping to diag_1, diag_2, diag_3
for col in ["diag_1", "diag_2", "diag_3"]:
    data[col + "_category"] = data[col].apply(map_icd9_category)

# Combine all diagnosis categories into one column
all_diags = pd.concat([
    data["diag_1_category"],
    data["diag_2_category"],
    data["diag_3_category"]
])

# Count frequencies
category_counts = all_diags.value_counts().reset_index()
category_counts.columns = ["Category", "Frequency"]

# Plot distribution
plt.figure(figsize=(12,6))
sns.barplot(data=category_counts, x="Category", y="Frequency", palette="viridis")
plt.xticks(rotation=45, ha="right")
plt.title("Distribution of Diagnosis Categories (ICD-9)")
plt.xlabel("Diagnosis Category")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# 5. Frequency Table
# Create a DataFrame from the three diagnosis category columns
diag_df = pd.DataFrame({
    'diag_1': data['diag_1_category'],
    'diag_2': data['diag_2_category'],
    'diag_3': data['diag_3_category']
})

# Calculate the value counts for each diagnosis category
diag_table = diag_df.stack().value_counts(normalize=True) * 100
print("Percentage Distribution of Diagnoses:")
print(diag_table.round(2))
```

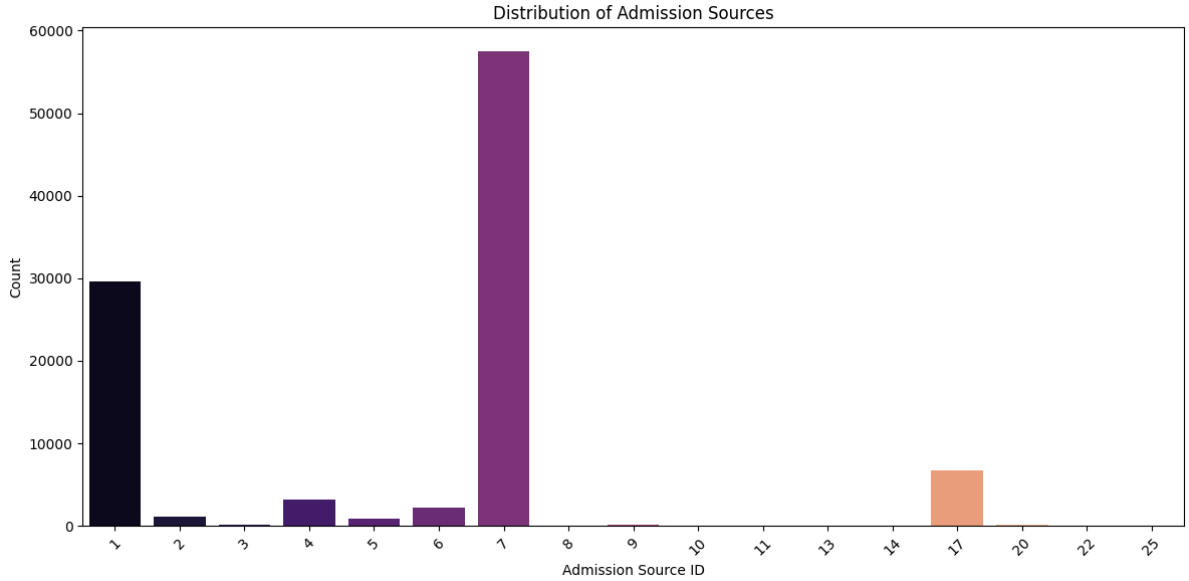


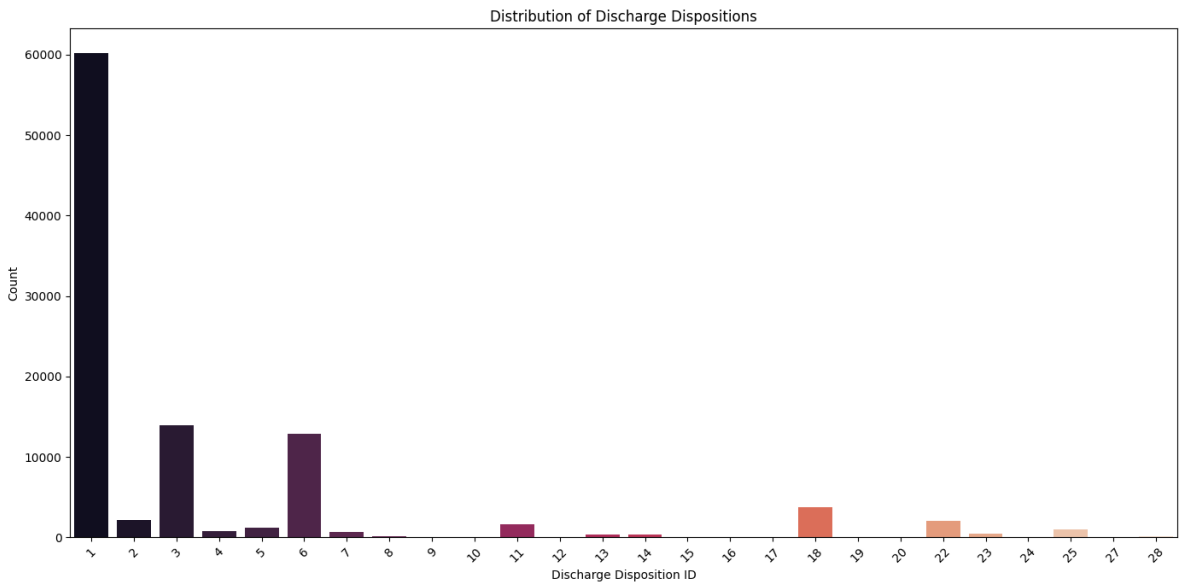

```
Percentage Distribution of Diagnoses:
Circulatory                          30.01
Endocrine/Metabolic or Diabetic      19.25
Respiratory                           8.99
Genitourinary                         6.35
Symptoms/Ill-defined                  5.50
Digestive                             5.48
Injury/Poisoning                      3.72
Supplementary                         3.03
Musculoskeletal                       2.83
Skin                                  2.82
Mental Disorders                      2.64
Neoplasms                             2.57
Infectious Diseases                   2.15
Blood Disorders                       2.14
Nervous System                        1.40
Unknown                               0.59
Pregnancy                             0.46
Congenital Anomalies                  0.08
Name: proportion, dtype: float64
```

g) Explore the distribution of patients across admission types, sources, and discharge dispositions

```
# task 7: Explore the distribution of patients across admission types, sources, and discharge dispositions
# List of columns to visualize
columns_to_visualize = ['admission_type_id', 'admission_source_id', 'discharge_disposition_id']

# Loop through each column and generate the count plot
for col_name in columns_to_visualize:
    # Define plot parameters based on the column name
    if col_name == 'admission_type_id':
        title = 'Distribution of Admission Types'
        xlabel = 'Admission Type ID'
        palette = 'viridis'
        figsize = (10, 6)
    elif col_name == 'admission_source_id':
        title = 'Distribution of Admission Sources'
        xlabel = 'Admission Source ID'
        palette = 'magma'
        figsize = (12, 6)
    elif col_name == 'discharge_disposition_id':
        title = 'Distribution of Discharge Dispositions'
        xlabel = 'Discharge Disposition ID'
        palette = 'rocket'
        figsize = (14, 7)
    else:
        # Skip if the column name is not in the predefined list
        continue

    # Set the figure size for better readability
    plt.figure(figsize=figsize)
    # Create a count plot
    sns.countplot(data=data, x=col_name, palette=palette)
    # Add a title to the plot
    plt.title(title)
    # Add x-axis label
    plt.xlabel(xlabel)
    # Add y-axis label
    plt.ylabel('Count')
    # Rotate x-axis labels for better visibility
    plt.xticks(rotation=45)
    # Adjust layout to prevent labels from being cut off
    plt.tight_layout()
    # Display the plot
    plt.show()
```



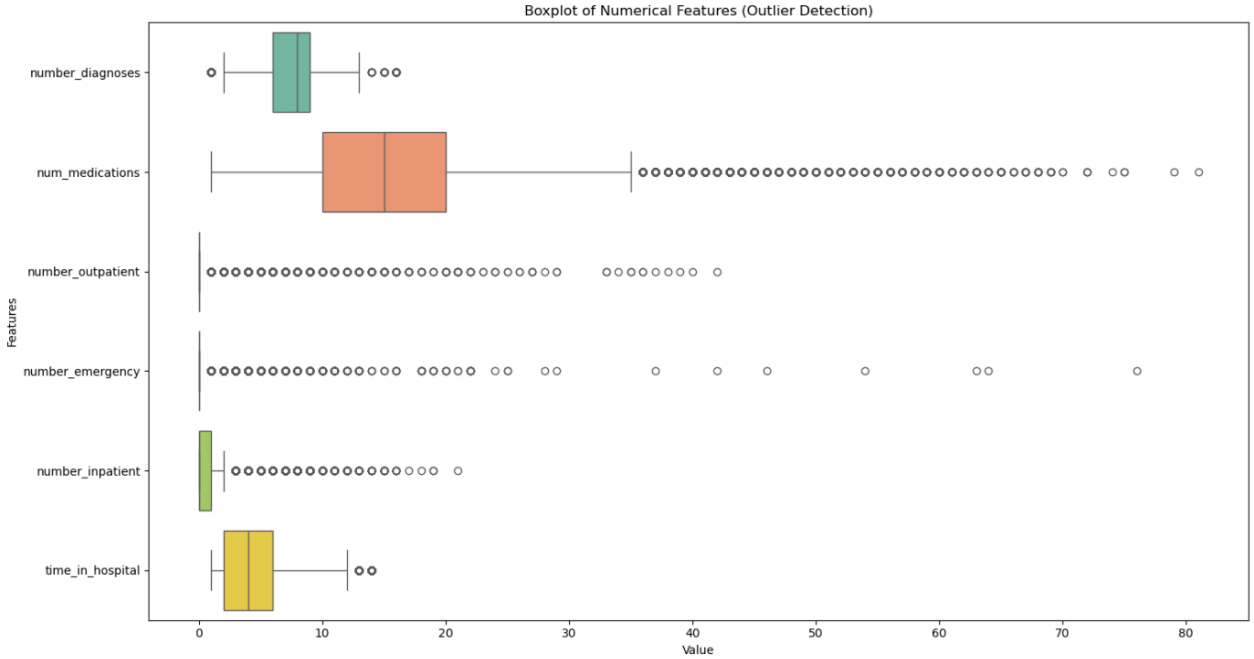




h) Identify and visualize any outliers in the dataset, especially in numerical features

```python
# Task 8: • Identify and visualize any outliers in the dataset, especially in numerical features

# Select numerical columns
num_cols = ['number_diagnoses', 'num_medications', 'number_outpatient', 'number_emergency', 'number_inpatient', 'time_in_hospital']
# Step 1: Detect outliers using IQR
outlier_summary = {}
for col in num_cols:
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[col] < lower_bound) | (data[col] > upper_bound)][col]
    outlier_summary[col] = len(outliers)

# Print outlier counts per column
print("Outlier counts per numerical feature:")
for col, count in outlier_summary.items():
    print(f"{col}: {count}")

# Step 2: Visualize with boxplots
plt.figure(figsize=(15, 8))
sns.boxplot(data=data[num_cols], orient="h", palette="Set2")
plt.title("Boxplot of Numerical Features (Outlier Detection)")
plt.xlabel("Value")
plt.ylabel("Features")
plt.tight_layout()
plt.show()
```

```
Outlier counts per numerical feature:
number_diagnoses: 281
num_medications: 2557
number_outpatient: 16739
number_emergency: 11383
number_inpatient: 7049
time_in_hospital: 2252
```



i) Write an analysis report on performing exploratory data analysis (EDA) using Python in the context of building a readittance detection system for the healthcare management system

Exploratory Data Analysis (EDA) Report: Readmission Detection System

The Exploratory Data Analysis (EDA) conducted on the `diabetic_data.csv` dataset, which aims to lay the groundwork for building a readmission detection system in healthcare management. The primary goal of this EDA was to understand the dataset's structure, identify key characteristics, detect patterns, and uncover potential anomalies that could influence patient readmission.

1) **Data Overview and Initial Inspection**
   Shape: The DataFrame contains 101,766 entries and 50 columns.
   Data Types: The dataset comprises 13 numerical (int64) and 37 categorical (object) features. Key numerical features include `time_in_hospital`, `num_lab_procedures`, `num_medications`, `number_outpatient`, `number_emergency`, `number_inpatient`, and `number_diagnoses`.
   Missing Values: Initial inspection revealed missing values, particularly in `max_glu_serum` and `A1Cresult`, which will require careful handling during feature engineering. Other columns like `weight` also show a high proportion of missing values represented as '?'.

2) **Summary Statistics**
   Descriptive statistics (`data.describe()`) provided insights into the central tendency, dispersion, and shape of the numerical features. For example, `time_in_hospital` ranges from 1 to 14 days, with an average of approximately 4.4 days. `num_medications` shows a wide range (1 to 81), indicating variability in medication regimens.

3) **Distribution of Categorical Features**

Race: The distribution of `race` shows 'Caucasian' as the predominant group, followed by 'AfricanAmerican'. Other racial categories have significantly fewer observations.

Gender: The `gender` distribution is relatively balanced, with slightly more 'Female' patients than 'Male'. The presence of 'Unknown/Invalid' gender entries needs to be addressed.

Age: Patients are distributed across various `age` groups, with '70-80)' and '60-70)' being the most frequent. The ordered nature of this feature is important for analysis.

4) **Relationship between Readmission Status and Age**

Analysis of `readmitted` status across `age` groups revealed variations in readmission rates. Higher readmission counts were observed in older age groups, particularly '70-80)' and '60-70)', suggesting that age is a significant factor in readmission risk.

5) **Correlations Between Numerical Features**

A heatmap of the correlation matrix for numerical features highlighted some interesting relationships:

- `time_in_hospital` shows moderate positive correlations with `num_lab_procedures` and `num_medications`, which is expected as longer hospital stays might involve more tests and medications.
- `number_inpatient`, `number_emergency`, and `number_outpatient` generally have weak positive correlations with other features but are important indicators of past healthcare utilization.

6) **Medication Analysis**

Medication Changes (`change`): The distribution of `change` (whether there was a change in medication) shows a significant number of patients undergoing medication adjustments during their hospital stay.

Diabetes Medication (`diabetesMed`): The majority of patients in the dataset were on diabetes medication, indicating the focus of the dataset on diabetic patients.

Number of Medications (`num_medications`): A histogram revealed that `num_medications` generally follows a right-skewed distribution, with most patients taking a moderate number of medications, but a long tail extending to a high number of medications. This suggests that a few patients are on complex medication regimens.

7) **Distribution of Diagnosis Categories**

Individual Diagnosis Columns (`diag_1`, `diag_2`, `diag_3`): Count plots for `diag_1`, `diag_2`, and `diag_3` (displaying the top 10 for each) showed a concentration of certain diagnosis codes. Circulatory, respiratory, and diabetes-related codes frequently appeared as primary, secondary, and tertiary diagnoses.

Combined Diagnosis Categories: A combined view of the top 15 diagnosis categories across all three diagnosis columns highlighted the most prevalent health issues in the patient population. Diseases of the circulatory system (e.g., specific heart conditions), diabetes, and certain symptoms/signs were dominant.

8) Admission and Discharge Patterns

Admission Types: The distribution of `admission_type_id` indicates that 'Emergency' and 'Urgent' admissions are the most common, followed by 'Elective'. This highlights the acute nature of many patient encounters.

Admission Sources: `admission_source_id` shows that most patients are admitted through 'Emergency Room' or 'Referral'.

Discharge Disposition: `discharge_disposition_id` reveals that the most frequent disposition is 'Discharged to home', but a notable portion of patients are discharged to skilled nursing facilities or experience other types of discharge, some of which may be related to readmission risk.

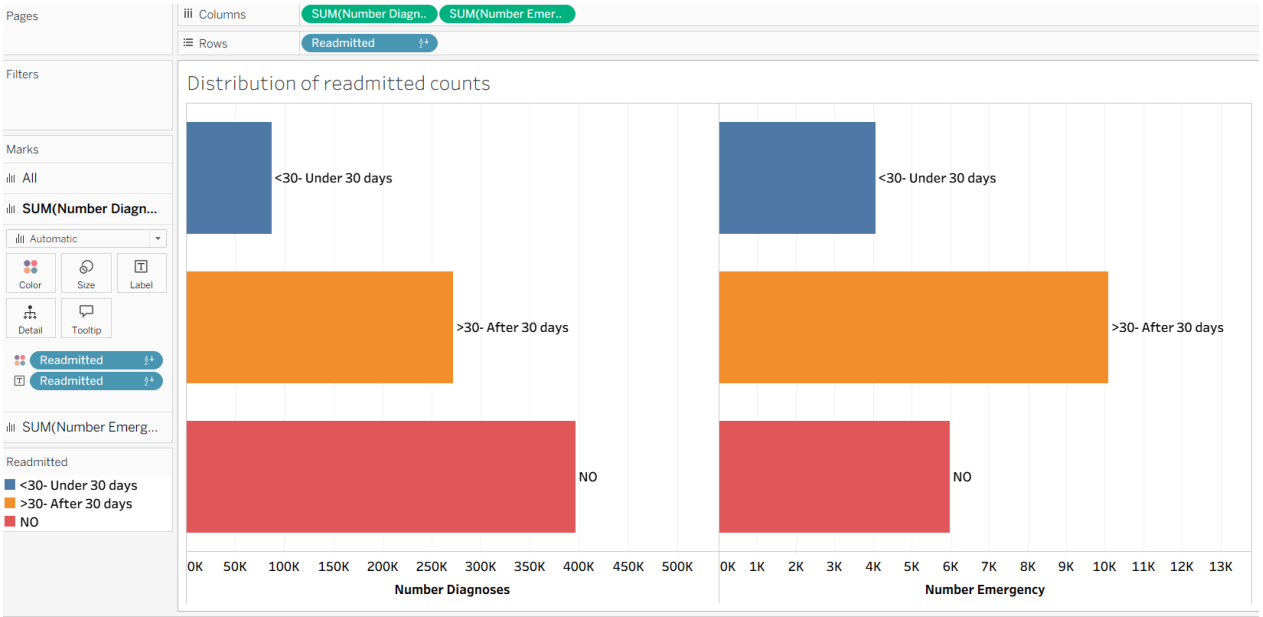
Conclusion

This EDA has provided a robust understanding of the diabetic patient dataset. We have identified key demographic, clinical, and administrative factors that may influence readmission. Features such as `age`, `time_in_hospital`, `num_medications`, and certain `diagnosis` categories appear to be significant.

5. Tableau Tasks:

Interactive Dashboard Design in Tableau:

- Show the distribution of readmitted counts of diagnoses and emergency cases through bar graphs
  - Choose the **Text file** in the Connect pane and upload the **diabetic_data.csv** dataset
  - Open a new worksheet
  - Drag **Number Diagnoses** and **Number Emergency** into **Columns** and **Readmitted** to **Rows**
  - Drag **Readmitted** to **Label**



❖ Create a visual capturing age bucket count by readmitted

- Open a new worksheet, click on **Analysis,** and choose **Create Calculated Field**
- Name the calculated field as **Age Numeric**, and use the formula: **INT([Age])**, and click on **Apply** and **OK** o Again, click on **Analysis** and choose **Create Calculated Field**

Transformation of "Age" from Categorical to Numerical

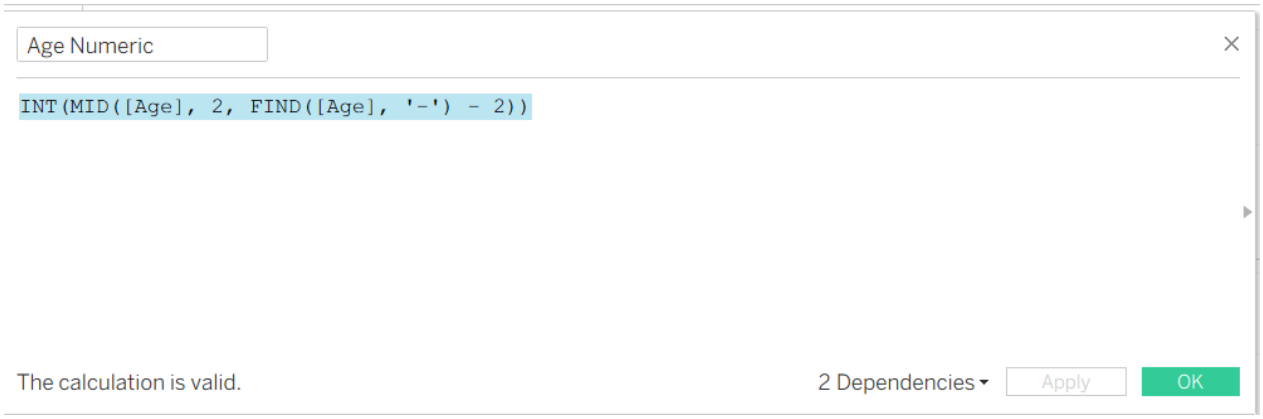
In the original diabetic_data.csv dataset, the Age column is recorded as a categorical string in 10-year buckets (e.g., [0-10), [10-20), [50-60)). To perform statistical analysis or use this variable in a machine learning model, these buckets must be converted into a numerical format.

The initial formula =INT([Age]) resulted in an error because the data starts with a non-numeric character ([). Traditional integer conversion cannot process the brackets or the range dash directly.

To extract a usable number, we implemented a string manipulation formula. This logic identifies the start of the age range by locating the dash (-) and extracting the characters between the initial bracket and that dash.

The Corrected Formula:

Age Numeric = INT(MID([Age], 2, FIND('-', [Age]) - 2))



- Name the calculated field as **Age Bucket**, use this formula:

IF [Age Numeric] <= 10 THEN "0-10"

ELSEIF [Age Numeric] <= 20 THEN "11-20"

ELSEIF [Age Numeric] <= 30 THEN "21-30"

ELSEIF [Age Numeric] <= 40 THEN "31-40"

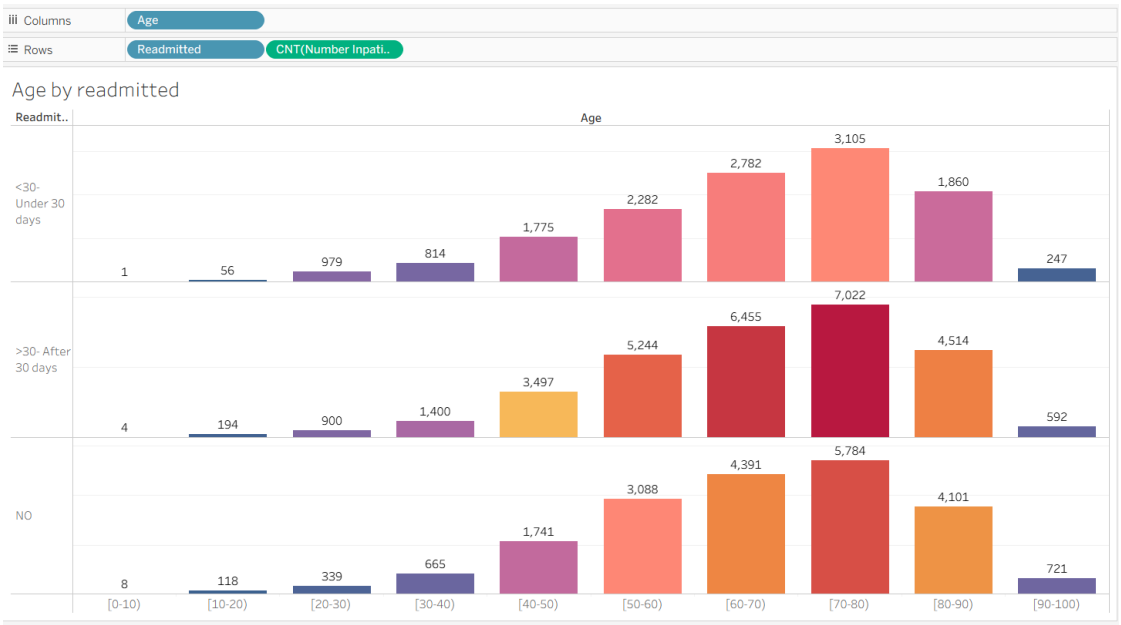
ELSEIF [Age Numeric] <= 50 THEN "41-50"

ELSE "51+"

END

and click on **Apply** and **OK**

- Drag **Age Bucket** to **Columns** and **Readmitted** and **Number Inpatient** to **Rows**
- Right-click on **Number Inpatient** and change the measure from sum to count
- Drag **Number Inpatient** to **Color** and **Label**

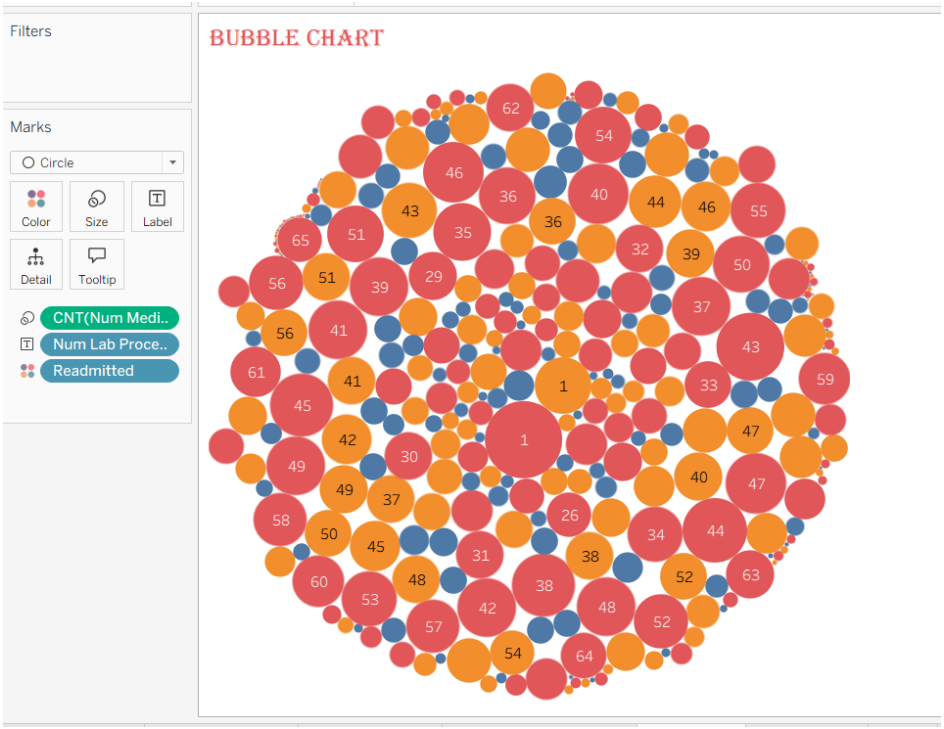


❖ Visualize the number of medications, procedures, diagnoses, emergencies, time in the hospital by readmitted

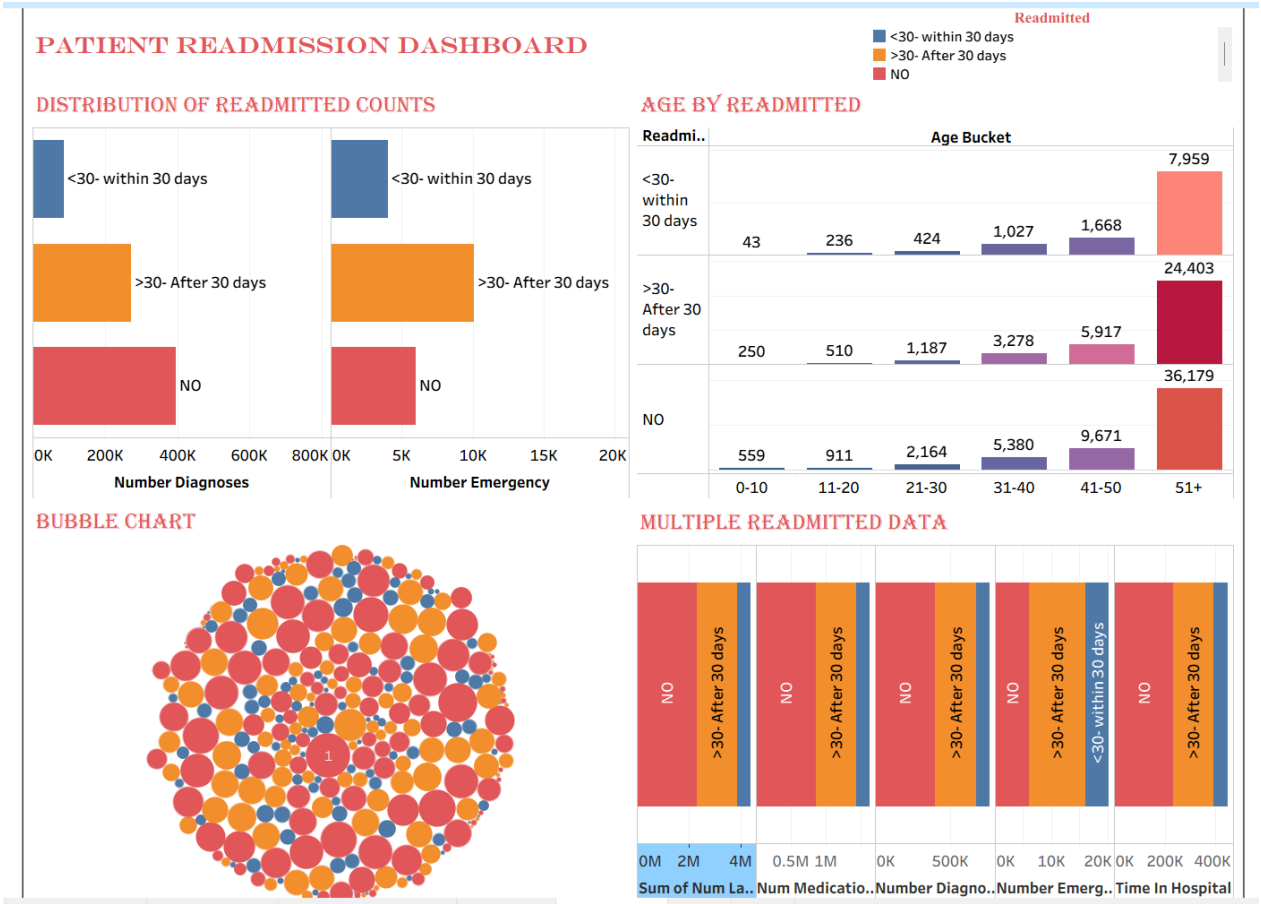
- Drag **Readmitted** to **Columns** and **Num Medications**, **Num Procedures**, **Number Diagnoses**, **Number Emergency** and **Time In Hospital** to **Rows**
- Select the stacked bar chart from show me
- Interchange the rows and columns for a better visualization

❖ Create a bubble chart showing the number of medications, Lab procedures and Diagnoses and use Readmitted as color code

- Drag Num **Medications** to **Columns** and **Num Lab Procedures** to **Rows**
- Drag **Number Diagnoses** to **Size**
- Drag **Readmitted** to **Color**



❖ Create a dashboard with all the visualizations
- Click on the dashboard icon at the bottom
- Drag **Readmitted**, **Age Bucket**, **Multiple Readmitted Data**, and **Bubble Chart** to the dashboard area



Key Findings

- **The Age Factor:** Analysis of the derived Age Bucket column reveals that readmission risk is not uniform. Mid-to-late age groups show a higher frequency of inpatient stays, which correlates strongly with an increased likelihood of returning to the facility.
- **Clinical Complexity:** The data indicates that patients with a high volume of medications (Num Medications) combined with multiple laboratory procedures (Num Lab Procedures) are at a heightened risk. These patients represent complex cases where post-discharge care instructions are critical.
- **Resource Utilization:** There is a notable correlation between previous "Emergency" visits and subsequent readmission. Patients who frequently utilize emergency services are significantly more likely to be readmitted within the 30-day window.

Strategic Recommendations

To achieve the mission of reducing healthcare costs and improving patient outcomes, **ReadmitGuard** recommends the following interventions:

- **Targeted Discharge Planning:** Implement an "Enhanced Discharge Protocol" for patients identified in the high-risk bubble chart clusters (those with high medication and diagnosis counts).
- **Age-Specific Outreach:** Develop proactive follow-up programs for patients in the 51+ age category, as this demographic demonstrates the highest strain on inpatient resources.
- **Integrated Monitoring:** Use the Tableau dashboard as a real-time clinical tool to flag patients during their stay who meet the "high-risk" criteria before they are discharged.

Final Statement

This project demonstrates that data-driven intelligence is the key to transitioning from reactive to proactive healthcare. By utilizing the insights from this report, the facility can not only reduce financial penalties associated with readmissions but, more importantly, ensure a higher standard of recovery and long-term health for every patient.